

Cours complet Git et GitHub

De la création du projet au déploiement sur GitHub

Guide pratique

1^{er} octobre 2026

Git + GitHub + VS Code + projets Flutter / Web / Java / etc.

Table des matières

1	Introduction	4
2	Git et GitHub : différence	4
3	Installation de Git	5
4	Configuration initiale de Git	5
5	Créer un projet local	5
6	Initialiser Git dans le projet	6
7	Le fichier .gitignore	6
8	Vérifier les fichiers	7
9	Ajouter les fichiers	7
10	Créer le premier commit	8
11	Comprendre le fonctionnement de Git	8
12	Créer le dépôt GitHub	9
13	Renommer master en main	9
14	Connecter le projet local à GitHub	9
15	Premier push	10

16 Erreur : rejected - fetch first	10
17 Pourquoi –allow-unrelated-histories ?	10
18 Erreur : CONFLICT (add/add)	11
19 Comprendre un conflit Git	11
20 Résoudre un conflit	12
21 Erreur : remote origin already exists	12
22 Erreur : repository not found	12
23 Erreur d'authentification	13
24 Erreur : failed to push some refs	13
25 Erreur : non-fast-forward	13
26 Erreur : MERGE_H EADexists	14
27 Annuler une fusion en cours	14
28 Mettre à jour un projet existant	14
29 Récupérer les modifications de GitHub	15
30 git fetch et git pull	15
31 Consulter l'historique	15
32 Connaître le commit actuel	16
33 Voir les différences	16
34 Annuler une modification non commitée	16
35 Retirer un fichier du staging	17
36 Supprimer un fichier du suivi Git	17
37 Cas important : build ajouté par erreur	17
38 Les branches	17
39 Supprimer une branche	18
40 Envoyer une nouvelle branche sur GitHub	18
41 Fusionner une branche	18
42 Le fichier README.md	19

43 Bonnes pratiques pour les commits	19
44 Ne jamais mettre les secrets sur GitHub	20
45 Si un secret a déjà été envoyé	20
46 Git status : commande la plus importante	21
47 Workflow complet pour un nouveau projet	21
47.1 Étape 1 : ouvrir le projet	21
47.2 Étape 2 : initialiser Git	21
47.3 Étape 3 : créer .gitignore	21
47.4 Étape 4 : vérifier	22
47.5 Étape 5 : ajouter	22
47.6 Étape 6 : premier commit	22
47.7 Étape 7 : utiliser main	22
47.8 Étape 8 : créer GitHub	22
47.9 Étape 9 : connecter	22
47.10 Étape 10 : vérifier	22
47.11 Étape 11 : push	22
48 Workflow recommandé pour les projets suivants	22
49 Cas réel : StyleAI	23
50 Commandes essentielles à mémoriser	23
51 Tableau de dépannage	24
52 Commande de diagnostic	25
53 Quand utiliser git push -force ?	25
54 Cloner un projet GitHub existant	25
55 GitHub et collaboration	26
56 Bonnes pratiques générales	26
57 Workflow final à retenir	26
58 Checklist avant un premier push	27
59 Conclusion	27

1 Introduction

Git et GitHub sont deux outils différents mais complémentaires.

- **Git** est un système de contrôle de version installé sur l'ordinateur.
- **GitHub** est une plateforme en ligne permettant d'héberger des dépôts Git.

Git permet notamment de :

- sauvegarder les différentes versions d'un projet ;
- revenir à une ancienne version ;
- travailler avec plusieurs personnes ;
- créer des branches ;
- suivre les modifications ;
- annuler certaines modifications ;
- conserver un historique du développement.

GitHub permet notamment de :

- stocker le projet en ligne ;
- partager le code ;
- collaborer avec d'autres développeurs ;
- présenter des projets dans un portfolio ;
- utiliser les fonctionnalités GitHub comme Issues, Pull Requests et Actions.

2 Git et GitHub : différence

Git	GitHub
Installé localement	Service en ligne
Contrôle les versions	Héberge les dépôts
Fonctionne hors ligne	Nécessite Internet pour synchroniser
Commandes dans le terminal	Interface Web + Git
Historique local	Copie distante du dépôt

Information

Un projet peut utiliser Git sans GitHub. En revanche, GitHub utilise Git pour gérer les versions du code.

3 Installation de Git

Sous Windows, télécharger Git depuis :

<https://git-scm.com/>

Après installation, ouvrir CMD, PowerShell ou Git Bash.

Vérifier l'installation :

```
git --version
```

Exemple :

```
git version 2.x.x
```

Si la commande est inconnue, Git n'est probablement pas installé ou son chemin n'est pas configuré dans le système.

4 Configuration initiale de Git

Avant le premier projet, configurer le nom et l'adresse e-mail.

```
git config --global user.name "Votre Nom"
```

Puis :

```
git config --global user.email "votre-email@example.com"
```

Vérifier :

```
git config --global --list
```

On peut également vérifier individuellement :

```
git config --global user.name  
git config --global user.email
```

Important

L'adresse e-mail utilisée dans Git n'est pas obligatoirement l'adresse avec laquelle vous vous connectez à GitHub, mais il est généralement préférable d'utiliser une adresse associée à votre compte GitHub.

5 Créer un projet local

Supposons que le projet se trouve ici :

```
C:\Users\HP\style_ai
```

Se placer dans le dossier :

```
cd C:\Users\HP\style_ai
```

Vérifier les fichiers :

```
dir
```

Avec PowerShell, on peut également utiliser :

```
Get-ChildItem
```

6 Initialiser Git dans le projet

Dans le dossier du projet :

```
git init
```

Git crée alors un dossier caché :

```
.git
```

Ce dossier contient l'historique et les informations du dépôt.

Vérifier :

```
git status
```

Exemple :

```
On branch master

No commits yet

Untracked files:
  ...
```

7 Le fichier .gitignore

Le fichier `.gitignore` indique à Git quels fichiers ne doivent pas être suivis.

Il est extrêmement important.

Par exemple, pour Flutter :

```
.dart_tool/
.flutter-plugins
.flutter-plugins-dependencies
.pub-cache/
.pub/
build/
coverage/

android/.gradle/
android/local.properties
android/**/build/

ios/Pods/
ios/.symlinks/
ios/Flutter/ephemeral/

.env
.env.*
```

```
!.env.example  
  
.idea/  
.vscode/  
  
node_modules/
```

Il ne faut généralement pas envoyer :

- les fichiers générés ;
- les fichiers temporaires ;
- les mots de passe ;
- les clés privées ;
- les variables secrètes ;
- les dossiers de build ;
- les dépendances générées automatiquement.

Erreur

Ne jamais mettre une clé API, un mot de passe, une clé privée ou un fichier contenant des secrets dans un dépôt GitHub public.

8 Vérifier les fichiers

Utiliser :

```
git status
```

Git indique :

- les fichiers suivis ;
- les fichiers modifiés ;
- les nouveaux fichiers ;
- les fichiers ignorés ;
- les fichiers prêts à être commités.

9 Ajouter les fichiers

Pour ajouter tous les fichiers :

```
git add .
```

Puis vérifier :

```
git status
```

Les fichiers doivent apparaître sous :

```
Changes to be committed:
```

On dit qu'ils sont dans la **staging area**.

10 Créer le premier commit

Créer le commit :

```
git commit -m "Initial commit"
```

Pour un projet StyleAI :

```
git commit -m "Initial commit - StyleAI frontend"
```

Vérifier :

```
git status
```

Résultat attendu :

```
nothing to commit, working tree clean
```

11 Comprendre le fonctionnement de Git

Le fonctionnement peut être représenté ainsi :

Fichiers du projet

↓

git add

↓

Staging Area

↓

git commit

↓

Repository local

↓

git push

↓

GitHub

12 Créer le dépôt GitHub

Se connecter à GitHub :

<https://github.com/>

Cliquer sur :

New repository

Choisir un nom, par exemple :

```
style-ai
```

Lorsqu'un projet local existe déjà, il est généralement plus simple de créer un dépôt GitHub sans ajouter automatiquement :

- README;
- .gitignore;
- License.

Important

Si GitHub crée déjà un README ou un autre fichier, le dépôt distant possède alors un commit que le dépôt local n'a pas. Cela peut provoquer une erreur lors du premier push.

13 Renommer master en main

Les nouveaux projets utilisent généralement `main` comme branche principale.

Commande :

```
git branch -M main
```

Vérifier :

```
git status
```

On doit voir :

```
On branch main
```

14 Connecter le projet local à GitHub

Ajouter le dépôt distant :

```
git remote add origin https://github.com/USERNAME/REPOSITORY.git
```

Exemple :

```
git remote add origin https://github.com/fatihabenbihirsid-dotcom/style-ai.git
```

Vérifier :

```
git remote -v
```

Résultat :

```
origin https://github.com/fatihabenbihirsid-dotcom/style-ai.git (fetch)
origin https://github.com/fatihabenbihirsid-dotcom/style-ai.git (push)
```

15 Premier push

Si le dépôt GitHub est réellement vide :

```
git push -u origin main
```

Le paramètre `-u` établit le suivi entre :

```
main
```

et :

```
origin/main
```

Après cela, les prochains push peuvent généralement être effectués simplement avec :

```
git push
```

16 Erreur : rejected - fetch first

Erreur typique :

```
! [rejected] main -> main (fetch first)

error: failed to push some refs
hint: Updates were rejected because the remote
contains work that you do not have locally.
```

Cela signifie que GitHub contient déjà un ou plusieurs commits que le dépôt local ne possède pas.

Cela arrive notamment lorsque le dépôt GitHub a été créé avec un README.

Solution :

```
git pull origin main --allow-unrelated-histories
```

Puis :

```
git push -u origin main
```

17 Pourquoi `--allow-unrelated-histories` ?

Si le dépôt local et le dépôt GitHub ont été initialisés séparément, Git peut considérer qu'ils ont deux historiques différents.

Exemple :

Dépôt local

$$A \rightarrow B$$

et :

GitHub

$$X \rightarrow Y$$

Git doit être explicitement autorisé à fusionner ces deux historiques.

La commande :

```
git pull origin main --allow-unrelated-histories
```

permet cette fusion.

18 Erreur : CONFLICT (add/add)

Exemple :

```
CONFLICT (add/add): Merge conflict in .gitignore
```

```
Automatic merge failed; fix conflicts
and then commit the result.
```

Cela signifie que le même fichier existe des deux côtés.

Exemple classique :

```
.gitignore
```

Le fichier local contient une version et GitHub en contient une autre.

19 Comprendre un conflit Git

Git peut afficher :

```
<<<<<<< HEAD
VERSION LOCALE
=====
VERSION GITHUB
>>>>>>> origin/main
```

Signification :

- «<<<<< HEAD : début de la version locale ;
- ===== : séparation ;
- »>>>>> origin/main : version distante.

Il faut choisir ou combiner les deux versions.

Après résolution, il ne doit rester aucune ligne :

```
<<<<<<<
=====
>>>>>>>
```

20 Résoudre un conflit

Après avoir modifié le fichier :

```
git add .gitignore
```

Vérifier :

```
git status
```

Puis terminer la fusion :

```
git commit -m "Merge remote main and resolve conflict"
```

Enfin :

```
git push -u origin main
```

21 Erreur : remote origin already exists

Erreur :

```
error: remote origin already exists.
```

Cela signifie qu'un remote appelé origin existe déjà.

Vérifier :

```
git remote -v
```

Pour changer son URL :

```
git remote set-url origin https://github.com/USERNAME/REPOSITORY.git
```

Ou supprimer puis recréer :

```
git remote remove origin  
git remote add origin https://github.com/USERNAME/REPOSITORY.git
```

22 Erreur : repository not found

Exemple :

```
remote: Repository not found.  
fatal: repository not found
```

Vérifier :

1. le nom du compte GitHub ;
2. le nom du repository ;
3. l'URL ;
4. les permissions d'accès ;
5. l'authentification GitHub.

Vérifier l'URL :

```
git remote -v
```

23 Erreur d'authentification

GitHub n'accepte plus l'ancien système consistant à utiliser simplement le mot de passe du compte comme mot de passe Git dans de nombreux scénarios HTTPS.

Selon la configuration, Git peut utiliser :

- Git Credential Manager ;
- authentification via navigateur ;
- Personal Access Token ;
- SSH.

Pour un débutant sous Windows, Git Credential Manager est généralement pratique.

24 Erreur : failed to push some refs

Cette erreur est générale.

Exemple :

```
error: failed to push some refs to ...
```

Il faut lire les lignes situées juste avant l'erreur.

Cas fréquent :

```
rejected (fetch first)
```

Solution :

```
git pull origin main
```

Puis résoudre les éventuels conflits et faire :

```
git push
```

25 Erreur : non-fast-forward

Exemple :

```
! [rejected] main -> main (non-fast-forward)
```

Le dépôt distant possède des changements que le dépôt local ne possède pas.

Solution habituelle :

```
git pull origin main
```

Puis :

```
git push
```

Si Git demande de résoudre un conflit, le conflit doit être traité avant le push.

26 Erreur : `MERGE_HEAD` exists

Exemple :

```
You have not concluded your merge (MERGE_HEAD exists).
```

Cela signifie qu'une fusion est en cours.

Vérifier :

```
git status
```

Si des conflits sont indiqués :

1. ouvrir les fichiers concernés ;
2. résoudre les conflits ;
3. utiliser `git add` ;
4. créer le commit de merge.

Exemple :

```
git add .  
git commit -m "Resolve merge conflicts"  
git push
```

27 Annuler une fusion en cours

Si l'on souhaite abandonner un merge avant de le terminer :

```
git merge --abort
```

Puis vérifier :

```
git status
```

Important

Utiliser cette commande uniquement lorsque l'on souhaite réellement annuler la fusion en cours.

28 Mettre à jour un projet existant

Après avoir modifié le code :

```
git status
```

Ajouter les modifications :

```
git add .
```

Créer un commit :

```
git commit -m "Update StyleAI"
```

Envoyer sur GitHub :

```
git push
```

Workflow classique :

```
git status
git add .
git commit -m "Description des modifications"
git push
```

29 Récupérer les modifications de GitHub

Pour récupérer les modifications distantes :

```
git pull
```

Ou explicitement :

```
git pull origin main
```

Information

`git pull` récupère les changements distants puis tente de les intégrer dans la branche locale.

30 git fetch et git pull

`git fetch` récupère les informations du dépôt distant sans fusionner automatiquement.

```
git fetch origin
```

`git pull` fait généralement :

```
git fetch
+
git merge
```

Donc :

fetch = récupérer

pull = récupérer + intégrer

31 Consulter l'historique

Commande :

```
git log
```

Version compacte :

```
git log --oneline
```

Version pratique :

```
git log --oneline --decorate --graph --all
```

Exemple :

```
* d96e0a3 Merge remote main and resolve gitignore conflict
* ab539b1 Initial repository setup
* ...
```

32 Connaître le commit actuel

Utiliser :

```
git log -1
```

Ou :

```
git rev-parse HEAD
```

33 Voir les différences

Pour voir les modifications non ajoutées :

```
git diff
```

Pour voir les modifications déjà dans le staging :

```
git diff --staged
```

34 Annuler une modification non commitée

Pour un fichier précis :

```
git restore fichier.dart
```

Pour toutes les modifications non staged :

```
git restore .
```

Erreur

Cette commande peut supprimer des modifications locales non sauvegardées. Toujours vérifier avec `git status` avant de l'utiliser.

35 Retirer un fichier du staging

Si un fichier a été ajouté avec :

```
git add fichier.txt
```

mais ne doit finalement pas être commit :

```
git restore --staged fichier.txt
```

Le fichier reste sur l'ordinateur mais n'est plus dans le staging.

36 Supprimer un fichier du suivi Git

Pour supprimer un fichier du projet et de Git :

```
git rm fichier.txt
```

Pour conserver le fichier sur l'ordinateur mais ne plus le suivre :

```
git rm --cached fichier.txt
```

Pour un dossier :

```
git rm --cached -r dossier/
```

Cette commande est particulièrement utile lorsqu'un dossier de build a été ajouté par erreur.

37 Cas important : build ajouté par erreur

Exemple :

```
git rm --cached -r android/build
```

Puis ajouter une règle dans `.gitignore` :

```
android/**/build/
```

Ensuite :

```
git add .  
git commit -m "Remove generated build files"  
git push
```

38 Les branches

Une branche permet de travailler sur une version parallèle du projet.

Voir les branches :

```
git branch
```

Créer une branche :

```
git branch develop
```

Changer de branche :

```
git switch develop
```

Créer et changer directement :

```
git switch -c feature/login
```

Revenir à main :

```
git switch main
```

39 Supprimer une branche

Supprimer une branche locale :

```
git branch -d develop
```

Forcer la suppression :

```
git branch -D develop
```

Erreur

-D peut supprimer une branche contenant des commits qui ne sont pas fusionnés.

40 Envoyer une nouvelle branche sur GitHub

Exemple :

```
git switch -c feature/login
```

Après les modifications :

```
git add .  
git commit -m "Add login feature"
```

Puis :

```
git push -u origin feature/login
```

41 Fusionner une branche

Revenir sur main :

```
git switch main
```

Mettre main à jour :

```
git pull
```

Fusionner :

```
git merge feature/login
```

Puis :

```
git push
```

42 Le fichier README.md

Un fichier README présente le projet.

Il peut contenir :

- le nom du projet ;
- une description ;
- les fonctionnalités ;
- les technologies utilisées ;
- l'installation ;
- l'utilisation ;
- des captures d'écran ;
- les auteurs ;
- la licence.

Exemple :

```
# StyleAI

Application de gestion de garde-robe numrique.

## Technologies

- Flutter
- Dart

## Fonctionnalites

- Gestion des vetements
- Creation de tenues
- Suggestions de tenues
- Meteo
```

43 Bonnes pratiques pour les commits

Un commit doit expliquer clairement la modification.

Exemples :

```
Initial commit
Add authentication
Fix login validation
Add wardrobe screen
Update README
Fix responsive layout
```

```
Remove generated build files
Update dependencies
```

Éviter :

```
update
test
aaa
final
final final
new
changes
```

44 Ne jamais mettre les secrets sur GitHub

Exemples de données sensibles :

- mots de passe ;
- API keys ;
- tokens ;
- clés privées ;
- certificats privés ;
- fichiers contenant des identifiants ;
- secrets Firebase ;
- credentials cloud.

Utiliser plutôt :

```
.env
```

et l'ajouter au :

```
.gitignore
```

Exemple :

```
.env
.env.*
!.env.example
```

On peut fournir un modèle :

```
.env.example
```

sans mettre les vraies valeurs.

45 Si un secret a déjà été envoyé

Si une clé secrète a été publiée, supprimer simplement le fichier du projet ne suffit pas toujours.

La première action importante est de :

1. révoquer la clé ;
2. générer une nouvelle clé ;
3. modifier l'application ;
4. vérifier l'historique Git.

Erreur

Considérer une clé publiée comme compromise, même si le fichier a ensuite été supprimé.

46 Git status : commande la plus importante

Quand on ne sait plus quoi faire :

```
git status
```

Cette commande indique généralement :

- la branche actuelle ;
- les fichiers modifiés ;
- les fichiers staged ;
- les conflits ;
- les opérations Git en cours ;
- l'état de synchronisation avec le remote.

Information

Avant d'exécuter une commande Git importante, commencer par `git status`.

47 Workflow complet pour un nouveau projet

Voici la procédure recommandée lorsqu'on possède déjà un projet local.

47.1 Étape 1 : ouvrir le projet

```
cd C:\Users\HP\mon_projet
```

47.2 Étape 2 : initialiser Git

```
git init
```

47.3 Étape 3 : créer .gitignore

Créer le fichier :

```
.gitignore
```

47.4 Étape 4 : vérifier

```
git status
```

47.5 Étape 5 : ajouter

```
git add .
```

47.6 Étape 6 : premier commit

```
git commit -m "Initial commit"
```

47.7 Étape 7 : utiliser main

```
git branch -M main
```

47.8 Étape 8 : créer GitHub

Créer un dépôt portant le même nom.

47.9 Étape 9 : connecter

```
git remote add origin https://github.com/USERNAME/REPOSITORY.git
```

47.10 Étape 10 : vérifier

```
git remote -v
```

47.11 Étape 11 : push

```
git push -u origin main
```

48 Workflow recommandé pour les projets suivants

Après la première configuration :

```
git status  
git add .  
git commit -m "Description de la modification"  
git push
```

Si quelqu'un a modifié GitHub :

```
git pull
```

Puis :

```
git add .  
git commit -m "..."  
git push
```

49 Cas réel : StyleAI

Dans le projet StyleAI :

```
C:\Users\HP\style_ai
```

Le dépôt GitHub était :

```
https://github.com/fatihabenbihirsid-dotcom/style-ai
```

La procédure effectuée était :

```
git branch -M main
```

Puis :

```
git remote add origin https://github.com/fatihabenbihirsid-dotcom/style-ai.git
```

Le premier push a été refusé :

```
! [rejected] main -> main (fetch first)
```

La raison était que GitHub contenait déjà un historique.

La solution utilisée :

```
git pull origin main --allow-unrelated-histories
```

Un conflit est ensuite apparu dans :

```
.gitignore
```

Après résolution :

```
git add .gitignore
```

Puis :

```
git commit -m "Merge remote main and resolve gitignore conflict"
```

Enfin :

```
git push -u origin main
```

Le push a réussi.

50 Commandes essentielles à mémoriser

Commande	Utilité
<code>git init</code>	Initialiser Git
<code>git status</code>	Voir l'état du projet
<code>git add .</code>	Ajouter les modifications au staging
<code>git commit -m "..."</code>	Créer une version
<code>git push</code>	Envoyer vers GitHub
<code>git pull</code>	Récupérer les modifications
<code>git fetch</code>	Récupérer les informations distantes
<code>git clone URL</code>	Télécharger un dépôt GitHub
<code>git branch</code>	Voir les branches
<code>git switch main</code>	Changer de branche
<code>git switch -c nom</code>	Créer une branche
<code>git merge branche</code>	Fusionner une branche
<code>git log</code>	Voir l'historique
<code>git diff</code>	Voir les modifications
<code>git remote -v</code>	Voir les dépôts distants
<code>git remote add origin URL</code>	Ajouter un dépôt distant
<code>git remote set-url origin URL</code>	Modifier l'URL distante
<code>git restore fichier</code>	Annuler une modification locale
<code>git restore -staged fichier</code>	Retirer un fichier du staging
<code>git rm -cached fichier</code>	Arrêter de suivre un fichier
<code>git merge -abort</code>	Annuler un merge en cours

51 Tableau de dépannage

Erreur	Action principale
fetch first	<code>git pull origin main</code> <code>-allow-unrelated-histories</code>
non-fast-forward	Faire un <code>git pull</code> , résoudre les conflits si nécessaire, puis <code>git push</code>
remote origin already exists	<code>git remote -v</code> puis <code>git remote set-url origin URL</code>
repository not found	Vérifier l'URL, le compte et les permissions
MERGE_HEAD exists	Terminer le merge ou utiliser <code>git merge -abort</code>
CONFLICT	Ouvrir les fichiers concernés, résoudre les marqueurs, puis <code>git add</code>
failed to push	Lire le message précédent et vérifier <code>git status</code>
Fichier généré envoyé par erreur	Ajouter le fichier dans <code>.gitignore</code> et utiliser <code>git rm -cached</code>

Erreur	Action principale
Secret envoyé sur GitHub	Révoquer immédiatement le secret et en générer un nouveau
git: command not found	Installer Git et vérifier le PATH

52 Commande de diagnostic

Lorsqu'un problème apparaît, exécuter :

```
git status
git remote -v
git branch
git log --oneline --decorate --graph --all -10
```

Ces quatre commandes donnent beaucoup d'informations utiles.

53 Quand utiliser git push --force ?

La commande :

```
git push --force
```

peut réécrire l'historique distant.

Elle peut donc supprimer ou remplacer des commits présents sur GitHub.

Erreur

Ne pas utiliser `git push -force` comme solution automatique à une erreur de push.

Lorsque la réécriture est réellement nécessaire, une forme plus prudente est :

```
git push --force-with-lease
```

Mais cette commande doit être utilisée uniquement lorsque l'on comprend les conséquences sur l'historique distant.

54 Cloner un projet GitHub existant

Si le projet existe déjà sur GitHub et qu'on veut le récupérer :

```
git clone https://github.com/USERNAME/REPOSITORY.git
```

Puis :

```
cd REPOSITORY
```

Vérifier :

```
git status
```

Dans ce cas, il n'est normalement pas nécessaire de faire :

```
git init
```

car le dépôt Git est déjà présent.

55 GitHub et collaboration

Dans un projet à plusieurs développeurs, un workflow courant est :

```
git pull
git switch -c feature/ma-fonctionnalite
```

Développer puis :

```
git add .
git commit -m "Add ma fonctionnalite"
git push -u origin feature/ma-fonctionnalite
```

Ensuite, créer une Pull Request sur GitHub.

Après validation, la branche peut être fusionnée dans `main`.

56 Bonnes pratiques générales

1. Faire régulièrement des commits.
2. Donner des messages de commit explicites.
3. Ne jamais commit des secrets.
4. Utiliser un bon `.gitignore`.
5. Vérifier `git status` régulièrement.
6. Ne pas utiliser `git push -force` sans comprendre son effet.
7. Travailler avec des branches pour les nouvelles fonctionnalités.
8. Garder la branche principale stable.
9. Mettre régulièrement à jour le README.
10. Vérifier le projet avant chaque push.

57 Workflow final à retenir

Pour un nouveau projet local :

```
cd C:\Users\HP\mon_projet

git init

git add .

git commit -m "Initial commit"

git branch -M main
```

```
git remote add origin https://github.com/USERNAME/REPOSITORY.git  
git push -u origin main
```

Pour les mises à jour suivantes :

```
git status  
git add .  
git commit -m "Update project"  
git push
```

Si GitHub contient déjà un historique indépendant :

```
git pull origin main --allow-unrelated-histories
```

Puis résoudre les conflits si nécessaire :

```
git status  
git add .  
git commit -m "Resolve merge conflict"  
git push
```

58 Checklist avant un premier push

- Git est installé.
- Le nom Git est configuré.
- L'e-mail Git est configuré.
- Le projet contient un `.gitignore`.
- Aucun mot de passe n'est présent.
- Aucune clé API secrète n'est présente.
- `git status` a été vérifié.
- Les fichiers nécessaires ont été ajoutés avec `git add`.
- Le premier commit existe.
- La branche principale est `main`.
- Le remote `origin` est correct.
- Le repository GitHub est correct.
- Le premier push est effectué.
- `git status` est propre après le push.

59 Conclusion

Git permet de gérer l'historique d'un projet tandis que GitHub permet d'héberger et de partager ce dépôt en ligne.

Le cycle fondamental à retenir est :

Modifier → git status → git add → git commit → git push

Et pour récupérer le travail distant :

git pull

La commande la plus importante lorsqu'une erreur apparaît reste :

git status

Elle permet de comprendre dans quel état se trouve le dépôt avant de choisir la commande suivante.